

TriosSeconverter

Documentation technique complète

Auteur : Triomphe Tchounda

Version : 1.0

Date : Avril 2026

Langage : Python 3.10+

Objet : Conversion OCR de borniers electriques

-> Classeur Excel + Document Word uniques

1. Introduction et objectif du projet

TriosSeconverter est un outil Python qui automatise la transformation de tableaux de borniers électriques (images scannées dans un document Word) en fichiers numériques editables : un classeur Excel et un document Word.

Un bornier électrique est un tableau de connexions utilisé en électrotechnique industrielle. Chaque tableau liste les bornes (BORNE), leur couleur de câble (COULEUR), le signal associé (SIGNAL) et les jarretières (JARRETIERES).

Le problème résolu : traiter manuellement 200+ tableaux prendrait plusieurs jours. TriosSeconverter le fait en quelques minutes.

Pourquoi ce projet ?

- Les tableaux existent uniquement sous forme d'images dans des fichiers Word.
- Il n'est pas possible de chercher, filtrer ou modifier ces données.
- La saisie manuelle est longue, coûteuse en temps et source d'erreurs.
- L'OCR (reconnaissance optique de caractères) permet d'automatiser cette tâche.

Ce que le logiciel produit

- tous_les_borniers.xlsx : tous les tableaux sur une feuille Excel unique.
- tous_les_borniers.docx : tous les tableaux dans un document Word unique.
- Une interface graphique intuitive pour les non-développeurs.

2. Architecture du projet

Vue d'ensemble

Le projet est organisé en couches claires, chaque fichier ayant une responsabilité unique (principe de séparation des préoccupations) :

Interface (interface.py)	-> Fenêtre tkinter, interaction utilisateur
Orchestration (converter.py)	-> Pipeline complet avec callbacks
Génération (generer_classeur.py)	-> Assemblage Excel et Word
OCR (ocr_processor.py)	-> Extraction du texte depuis les images
Extraction (recuperer_image.py)	-> Décompression des images du .docx
Configuration (config.py)	-> Paramètres centralisés

Flux de traitement

Flux de données de haut en bas

```
Fichier Word (.docx)
|
v  recuperer_image.py - ImageExtractionPipeline
images/ (bornier_1.jpg, bornier_2.jpg, ...)
|
v  ocr_processor.py - BornierTableExtractor
résultats OCR [{headers, rows, metadata}, ...]
|
+--> generer_classeur.py - generer_excel()
|    tous_les_borniers.xlsx
|
+--> generer_classeur.py - generer_word()
|    tous_les_borniers.docx
```

Flux avec l'interface graphique

Communication thread worker ? thread UI

```
interface.py (thread principal - tkinter)
| clic sur 'Lancer'
v
threading.Thread(target=converter.run)
| callbacks : on_progress(pct, msg) / on_log(msg)
v
queue.Queue <--- messages de progression
| after(80ms, poll)
v
Mise à jour barre de progression + journal
```

3. Description détaillée de chaque fichier

config.py - Configuration centralisée

- Classe Config avec attributs de classe (constantes).
- TESSERACT_PATH : chemin vers l'exécutable Tesseract.
- OCR_LANGUAGE : langue OCR, 'fra' pour le français.
- IMAGES_FOLDER_NAME : nom du sous-dossier d'images.
- WORD_FILE : chemin par défaut du fichier Word source.
- Méthodes : get_output_folder(), get_word_file_path(), validate().
- Modifiez ce fichier pour adapter le logiciel à un nouveau projet.

recuperer_image.py - Extraction des images du document Word

- Un fichier .docx est une archive ZIP : on l'ouvre avec zipfile.
- Les images se trouvent dans word/media/ à l'intérieur du ZIP.
- ImageExtractor.extract_images() retourne [(bytes, extension), ...].
- ImageStorage.save_image() enregistre chaque image avec le nom bornier_N.
- ImageExtractionPipeline orchestre les deux classes ci-dessus.
- pipeline.run() retourne un dict {total, saved, errors, ...}.

ocr_processor.py - Reconnaissance optique et extraction structurée

- BornierTableExtractor est la classe centrale du projet.
- extract(image_path) -> dict {success, headers, rows, metadata}.
- Prétraitement : niveaux de gris + seuil Otsu + upscaling x2.
- Détection des colonnes : pixel X du centre de chaque en-tête.
- Affectation des mots : chaque mot va dans la colonne la plus proche.
- _fill_worksheet(ws, result, start_row) -> remplit une feuille Excel.
- _add_table_to_doc(doc, result) -> insère un tableau dans un doc Word.
- to_excel() et to_word() : wrappers pour fichiers individuels.

generer_classeur.py - Génération des fichiers combinés

- extraire_tous(images_dir) : boucle OCR sur toutes les images.
- generer_excel(results, extractor, path) : une feuille, tous les tableaux.
- generer_word(results, extractor, path) : un doc, saut de page entre borniers.
- barre(current, total, label) : affiche une barre ASCII dans le terminal.
- _cle_num(path) : tri numérique (bornier_1, bornier_2, ... bornier_100).
- 2 lignes vides entre chaque tableau dans Excel.
- Titre de section (heading niveau 2) avant chaque tableau dans Word.

convertir.py - Orchestrateur avec callbacks (pour l'interface graphique)

- Classe Convertir(word_file, output_dir, on_progress, on_log).
- run() exécute les 4 étapes et signale la progression via callbacks.
- on_progress(pct, msg) : pct dans [0.0, 1.0] pour la barre de progression.
- on_log(msg) : messages texte pour le journal de l'interface.
- Images stockées dans output_dir/images_borniers/ (chemin absolu).

- `_extraire_avec_progres()` : version de `extraire_tous()` avec callbacks.

interface.py - Interface graphique tkinter

- `TriosSeconverterApp` hérite de `tk.Tk`.
- `RoundedButton` : bouton Canvas à coins arrondis (dessin natif tkinter).
- Sélection de fichiers via `filedialog.askopenfilename / askdirectory`.
- `Converter.run()` s'exécute dans un thread daemon (`threading.Thread`).
- Les messages arrivent via `queue.Queue` et sont lus toutes les 80 ms.
- Boutons 'Excel', 'Word', 'Dossier' activés uniquement après succès.
- `os.startfile()` pour ouvrir les fichiers avec leur application par défaut.

run.py - Point d'entrée en ligne de commande

- Alternative à l'interface graphique pour les utilisateurs avancés.
- Exécute les 4 étapes avec affichage dans le terminal.
- Utilise `Config.IMAGES_FOLDER_NAME` comme dossier d'images.
- Les fichiers de sortie sont créés dans le répertoire courant.

4. Classes et méthodes clés

BornierTableExtractor (ocr_processor.py)

extract(image_path) -> dict

Méthode principale. Charge l'image, la prétraite, détecte les colonnes depuis les en-têtes, lit le texte ligne par ligne et classe chaque mot dans la bonne colonne.

Structure du dict retourné par extract()

```
result = {
    'success': True,
    'image_path': str(image_path),
    'headers': ['BORNE', 'COULEUR', 'SIGNAL', 'JARRETIERES'],
    'rows': [
        {'type': 'data', 'cells': ['A1', 'ROUGE', 'SIG1', '']},
        {'type': 'section', 'text': 'NOM DU CABLE : 24V'},
        {'type': 'footer', 'cells': ['M', 'T', 'I', ''], 'is_mti': True},
    ],
    'metadata': {'BORNIER': 'A', 'NO PLAN': '12'},
}
```

_detect_column_boundaries(data) -> list[int]

Cherche les mots-clés BORNE, COULEUR, SIGNAL, JARRETIERES dans les premières lignes du texte OCR. Retourne les positions x (pixels) du centre de chaque colonne. Cette approche est robuste car elle utilise les positions réelles des en-têtes, pas un clustering qui échoue quand une colonne est souvent vide.

_fill_worksheet(ws, result, start_row) -> int

Remplit une feuille openpyxl à partir de la ligne start_row. Retourne la première ligne libre après le pied de tableau. Gère trois types de lignes : data, section, footer. Les bordures des cellules fusionnées doivent être posées individuellement sur chaque cellule du périmètre (limitation openpyxl).

Pourquoi cette approche (erreur découverte en phase 3)

```
# Correction border merge : poser sur CHAQUE cellule du bord
ws.cell(row=r, column=1).border = Border(left=thin, right=thin,
                                          top=thin, bottom=ns)
ws.cell(row=r+1, column=1).border = Border(left=thin, right=thin,
                                          top=ns, bottom=thin)
```

Converter (converter.py)

Classe légère qui relie le pipeline de traitement à l'interface. Elle n'a aucune dépendance tkinter et peut donc être réutilisée dans des scripts ou des APIs. Le découplage repose sur les deux callables on_progress et on_log.

TriosSeconverterApp (interface.py)

Hérite de tk.Tk. L'UI est construite entièrement en code Python (pas de fichier .ui ou de ressource externe). Le thread worker ne touche jamais aux widgets directement - toutes les mises à jour passent par la queue, lue dans le thread principal par after().

5. Historique des erreurs et solutions

Double traitement OCR

[X] ERREUR	Cause : ImageExtractionPipeline avec enable_ocr=True ET generer_classeur qui relance BornierTableExtractor -> chaque image traitée deux fois, temps d'exécution doublé.
[OK]	Solution : Fixer enable_ocr=False dans l'appel à ImageExtractionPipeline dans run.py (et convertir.py). L'OCR est fait exclusivement par BornierTableExtractor via generer_classeur.

Bordures manquantes sur les cellules fusionnées

[X] ERREUR	Cause : Dans Excel, les bordures posées sur la cellule en haut à gauche d'une plage fusionnée ne s'affichent pas sur les autres bords de la zone fusionnée. openpyxl n'applique pas les bordures à toute la zone automatiquement.
[OK]	Solution : Poser les bordures manuellement sur chaque cellule du périmètre de la région fusionnée (top sur la ligne du haut, bottom sur la ligne du bas, left sur la colonne de gauche, right sur la droite).

PermissionError lors de la sauvegarde du fichier Word

[!] ATTENTION	Cause : PermissionError: [Errno 13] Permission denied: 'tous_les_borniers.docx' -> le fichier est ouvert dans Microsoft Word au moment où Python essaie de l'écrire.
[OK]	Solution : Fermer le fichier dans Word avant de relancer le script. L'interface affiche un message d'erreur clair dans ce cas.

Détection de colonnes par clustering (ancienne approche)

[!] ATTENTION	Cause : L'ancien code utilisait un clustering des positions x de tous les mots pour deviner les colonnes. Cela échouait souvent : la colonne COULEUR est fréquemment vide, donc sans point de clustering -> mauvaise détection.
[OK]	Solution : Nouvelle approche : lire les pixels x des mots-clés d'en-tête (BORNE, COULEUR, SIGNAL, JARRETIERES) directement depuis l'OCR des premières lignes. Robuste même si des colonnes sont vides.

Texte mal affecté aux colonnes

[X] ERREUR	Cause : Des mots appartenant à la colonne SIGNAL se retrouvaient dans COULEUR à cause de zones de colonnes mal délimitées.
[OK]	Solution : Utiliser les positions x réelles des en-têtes pour définir des zones par moitié : un mot appartient à la colonne dont le bord médian est le plus proche de son centre x.

6. Guide d'évolution du code

Comment ajouter une nouvelle colonne (ex: SECTION)

La liste des colonnes est implicite dans `BornierTableExtractor`. Pour ajouter une colonne :

- Ajouter le mot-clé dans la liste de mots-clés d'en-tête dans `_detect_column_boundaries()`.
- Ajuster la logique de fusion dans `_fill_worksheet()` si la nouvelle colonne participe aux fusions du pied de tableau.
- Mettre à jour les tests dans `test_nouveau_ocr.py`.

Comment changer la mise en forme Excel

- Les styles de bordures sont définis dans `_fill_worksheet()` via `openpyxl Border / Side`.
- `thin = Side(style='thin', color='000000')`
- `ns = Side(style=None)` <- pas de bordure
- Pour les lignes de section (NOM DU CABLE) : bordure en pointillés en haut (`style='dashed'`).

Comment supporter un autre format de Word

- `recuperer_image.py` -> `ImageExtractor.extract_images()` cherche `word/media/` dans le ZIP. Si les images sont ailleurs dans le ZIP, modifier le filtre `startswith()`.
- Si le fichier source est un `.doc` (ancien format), le convertir d'abord en `.docx` (python-docx ne supporte que `.docx`).

Comment améliorer la précision OCR

- Augmenter le facteur de zoom dans `_preprocess_image()` (actuellement x2).
- Essayer d'autres modes de page Tesseract : `--psm 6` (bloc de texte) ou `--psm 4` (colonne de texte).
- Ajouter une liste blanche de caractères dans la config Tesseract pour les tableaux de bornes (souvent alphanumériques).
- Ajouter un filtrage des artefacts OCR communs (0/O, l/1, etc.).

Comment déployer sur Linux / macOS

- Remplacer `TESSERACT_PATH` dans `config.py` par le chemin système (souvent `/usr/bin/tesseract`).
- Remplacer `os.startfile()` dans `interface.py` par `subprocess.run(['xdg-open', path])` sur Linux ou `['open', path]` sur macOS.
- `PyInstaller` supporte Linux et macOS avec les mêmes commandes (adapter le `.spec` si nécessaire).

7. Guide pour recoder TriosSeconverter depuis zéro

Étape 1 - Environnement de développement

- Installer Python 3.10+ depuis python.org (cocher 'Add to PATH').
- Installer Tesseract OCR (UB Mannheim) avec le pack de langue 'fra'.
- Créer un dossier de projet et un environnement virtuel :

Commandes d'initialisation

```
mkdir TriosSeconverter && cd TriosSeconverter
python -m venv venv
venv\Scripts\activate      # Windows
pip install python-docx openpyxl pytesseract opencv-python Pillow fpdf2
```

Étape 2 - Extraction des images (recuperer_image.py)

Un .docx est un ZIP. Ouvrez-le avec zipfile.ZipFile et extrayez tous les fichiers de word/media/. Nommez-les bornier_1.jpg, bornier_2.jpg...

```
import zipfile
from pathlib import Path

with zipfile.ZipFile('source.docx', 'r') as z:
    medias = [f for f in z.namelist() if f.startswith('word/media/')]
    for i, name in enumerate(medias, 1):
        data = z.read(name)
        ext = Path(name).suffix
        Path(f'images/bornier_{i}{ext}').write_bytes(data)
```

Étape 3 - OCR structuré (ocr_processor.py)

Pour chaque image, appliquez pytesseract avec image_to_data() qui retourne les positions x, y de chaque mot. Identifiez les colonnes par les en-têtes, puis affectez chaque mot à sa colonne.

```
import cv2, pytesseract

img = cv2.imread(str(path))
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, thresh = cv2.threshold(gray, 0, 255,
                          cv2.THRESH_BINARY + cv2.THRESH_OTSU)
big = cv2.resize(thresh, None, fx=2, fy=2,
                 interpolation=cv2.INTER_CUBIC)

data = pytesseract.image_to_data(
    big, lang='fra',
    config='--psm 6',
    output_type=pytesseract.Output.DICT
)
```

Étape 4 - Génération Excel (generer_classeur.py)

Créez un Workbook openpyxl, une feuille 'Borniers'. Pour chaque tableau de résultats, appelez _fill_worksheet() en incrémentant start_row (+ 2 lignes vides entre les tableaux).

```
from openpyxl import Workbook
```

```
wb = Workbook()
ws = wb.active
ws.title = 'Borniers'
row = 1
for result in ocr_results:
    row = extractor._fill_worksheet(ws, result, start_row=row)
    row += 2 # 2 lignes vides de séparation
wb.save('tous_les_borniers.xlsx')
```

Étape 5 - Génération Word (generer_classeur.py)

```
from docx import Document

doc = Document()
for i, result in enumerate(ocr_results):
    if i > 0:
        doc.add_page_break()
    doc.add_heading(result['metadata'].get('BORNIER',''), level=2)
    extractor._add_table_to_doc(doc, result)
doc.save('tous_les_borniers.docx')
```

Étape 6 - Interface graphique (interface.py)

- Créer une classe héritant de tk.Tk.
- Ajouter deux champs Entry + bouton 'Parcourir' pour le fichier Word et le dossier.
- Lancer la conversion dans threading.Thread(daemon=True).
- Communiquer avec le thread UI via queue.Queue et after(80, poll).
- N'JAMAIS modifier un widget tkinter depuis un thread worker.

Étape 7 - Packaging (optionnel)

```
pip install pyinstaller
pyinstaller --onedir --windowed --name TriosSeconverter interface.py
# Le résultat est dans dist/TriosSeconverter/
```

8. Dépendances et installation

Bibliothèques Python

python-docx	1.1.2+	Lire/écrire des fichiers .docx
openpyxl	3.1.5+	Créer et formater des fichiers Excel .xlsx
pytesseract	0.3.10+	Interface Python pour Tesseract OCR
opencv-python	4.8+	Prétraitement des images (seuillage, zoom)
Pillow	10+	Chargement d'images pour pytesseract
numpy	1.24+	Requis par opencv-python
fpdf2	2.7+	Génération PDF (ce document)

Tesseract OCR (logiciel externe)

Tesseract est le moteur OCR. Il doit être installé séparément sur la machine (pas un package Python). Le chemin est configuré dans config.py (TESSERACT_PATH).

- Windows : télécharger l'installateur UB-Mannheim depuis GitHub.
- Lors de l'installation, cocher le support de la langue 'French (fra)'.
- Chemin par défaut configuré : C:\Tesseract\TesseractOCR\tesseract.exe

Installation complète

```
# 1. Cloner / copier le projet dans un dossier
# 2. Double-cliquer sur install.bat
#    -> crée l'environnement venv
#    -> installe toutes les dépendances Python
#    -> crée TriosSeconvertir.bat (lanceur)
#    -> crée un raccourci sur le Bureau
# 3. S'assurer que Tesseract est installé séparément
# 4. Lancer via le raccourci ou TriosSeconvertir.bat
```